AraToken: Optimizing Arabic Tokenization with Normalization Pipeline and Language Extension for Qwen3

MARK KASHIRSKIY [1, 2], ARTIOM LIPINSKI [3] AND ILYA MAKAROV[4,5] (MEMBER, IEEE)
[1]HSE University, 101000 Moscow, Russia
[2]AI Talent Hub, ITMO University, 197101 Saint Petersburg, Russia
[3]Saint Petersburg State University, 199034 Saint Petersburg, Russia
[4]Artificial Intelligence Research Institute (AIRI), 105064 Moscow, Russia
[5]Trusted AI Center, RAS, 109004 Moscow, Russia

Corresponding author: Mark Kashirskiy (e-mail: 353056@niuitmo.ru).

The work was supported by a grant, provided by the Ministry of Economic Development of the Russian Federation (agreement dated June 20, 2025 No. 139-15-2025-011, identifier 000000C313925P4G0002).

ABSTRACT Arabic token fragmentation limits the text represented within a language model's token budget. We present AraToken, an Arabic normalization and SentencePiece Unigram tokenizer, and the Language Extension Pipeline (LEP), which adds its vocabulary to Qwen3. AraToken uses 44.4% fewer tokens per word than Qwen3 on 5,000 commonly normalized test documents. LEP initializes new embedding rows with a fixed average of compositional and character-ngram analogical estimates. We evaluate 78 controlled runs with three training seeds, two base-model sizes and explicit embedding and transformer trainability controls. Across six comparisons at matched nominal token budgets, LEP lowers canonical-Arabic bits per character by 0.035–0.123 relative to continued pretraining (CPT), while requiring 23–28% more trainer runtime. The fixed mixture improves on both components at the original 0.6B budget and at 1.7B, but its advantage over the analogical component is unresolved at the larger 0.6B budget. LEP achieves the highest mean accuracy on five Arabic tasks: 40.65%, versus 39.58% for CPT and 39.20% for the base, although the aggregate accuracy differences are not statistically resolved. On two English tasks, LEP averages 62.92%, improving on CPT's 58.33% while remaining below the unadapted base's 66.65%. The strongest findings are consistent Arabic compression and likelihood gains, accompanied by task-dependent accuracy gains and better English retention than CPT under the tested adaptation policies.

INDEX TERMS Arabic natural language processing, embedding initialization, large language models, tokenization, vocabulary extension

I. INTRODUCTION

The tokenizer decides how many tokens a language model spends on a text. Token counts can differ substantially across languages [1], [2]. Arabic illustrates this problem. Figure 1 shows how three tokenizers split the same words. The Qwen3 tokenizer cuts their libraries into four pieces and the borrowing technology into four more, while AraToken, the tokenizer we present here, keeps the stem of the first word intact and the second word whole. More tokens leave less text in a fixed-length token window; training and inference costs also depend on the model and execution setup.[1]

Arabic is hard to tokenize for two reasons. First, one word often carries a conjunction, a preposition, a stem and a possessive pronoun, all written without spaces [4]. Second, the same word can be spelled in several ways: Alif has four common variants, diacritics are optional, the decorative Tatweel stretches words, and writers mix Arabic-Indic and Western digits and punctuation [5], [6]. A tokenizer that ignores these properties spends its vocabulary on spelling variants of the same word.

Two lines of work address the problem. The first trains an Arabic tokenizer together with a new model, as in AraBERT [7] and the Arabic large language model (LLM) Jais [8]. The second extends the vocabulary of an existing model with Arabic tokens and continues training [9]–[11]. This reuses the pretrained model and requires an initialization for the added embeddings. Mean sub-

[1]AI assistance: OpenAI Codex [3]; see Acknowledgment.

token initialization averages the embeddings of the pieces produced by the old tokenizer [12]. WECHSEL [13] and FOCUS [14] instead use similarity in auxiliary embedding spaces. We study a fixed average of the sub-token estimate and a character-overlap estimate for Arabic vocabulary extension.

We train AraToken, an Arabic tokenizer with an explicit normalization pipeline, and add its vocabulary to Qwen3 [15] with the Language Extension Pipeline (LEP). LEP trains the new embedding rows and selected transformer layers. We compare mean sub-token initialization with a character-overlap initializer and their fixed average. The character-overlap initializer is a lexical proxy rather than an implementation of WECHSEL or FOCUS. We use bits per character (BPC) to compare models on the same declared character spans, with the initializer and trainable parameters identified for each configuration.

Our contributions are as follows:

- We present AraToken, a SentencePiece Unigram tokenizer with an Arabic normalization pipeline. On the normalized held-out sample, the full tokenizer needs 44.4% fewer tokens per word than Qwen3, and the pruned tokenizer needs 30.1% fewer than AraBERT at a smaller vocabulary size (Sections III-A, III-B, V-A and V-C).
- We present LEP, which adds the AraToken vocabulary to Qwen3 and supports comparisons between explicitly specified initialization and adaptation configurations (Sections III-C and V-D).
- We derive the expected squared distance of the sub-token mean from the donor mean under stated moment assumptions, and characterize the squared error of a fixed average relative to a reference row. We compare the component initializers and their average empirically (Sections III-C2 and V-E).
- We describe a cross-vocabulary evaluation protocol and release our tokenizer, training code, data splits and model checkpoints (Section IV-D; Appendices A and E; the Data and code availability statement).

The rest of the paper is organized as follows. Section II reviews related work. Section III describes AraToken and LEP. Section IV describes the experimental setup, and Section V reports the results. Section VI discusses them, and Section VII concludes the paper. The appendices give details of the evaluation protocol and further experiments.

II. RELATED WORK

We review subword tokenization, Arabic language models, and vocabulary extension with embedding initialization.[2]

A. SUBWORD TOKENIZATION

Most LLMs use subword tokenizers, which trade vocabulary size against sequence length. Byte Pair Encoding (BPE) [16] merges the most frequent pair of symbols until the vocabulary reaches its target size. WordPiece [17] chooses each merge by likelihood. The Unigram model [18] starts from a large vocabulary and removes the pieces that contribute least to the likelihood of the corpus. SentencePiece [19] implements BPE and Unigram directly on raw text, which suits languages where whitespace does not mark morpheme boundaries, and studies of morphologically rich languages report better compression for Unigram than for BPE [20]. Tokenizers built for English encode other languages at a higher cost [1], and the same text can need several times as many tokens in some languages as in English [2]. We measure this cost for Arabic on held-out text, distinguishing common external normalization from each tokenizer's internal preprocessing.

B. ARABIC NATURAL LANGUAGE PROCESSING AND LANGUAGE MODELS

Arabic natural language processing (NLP) has long dealt with rich morphology and dialectal variation [21]. CAMeL Tools [22] provides morphological analysis and orthographic normalization for Arabic. AraBERT [7], AraBART [23] and AraT5 [24] were pretrained with their own Arabic tokenizers, and recent Arabic LLMs such as Jais [8], AceGPT [25] and ALLaM [26] reach strong results on Arabic benchmarks after large-scale training on Arabic and English text. Two recent reports discuss tokenizer design and vocabulary adaptation for Arabic [9], [27]. AraToken is designed to supply vocabulary for adapting an existing model; the cost of that adaptation must be measured for its particular training configuration.

C. VOCABULARY EXTENSION AND EMBEDDING INITIALIZATION

Vocabulary extension adds tokens of a new language to a pretrained model and continues training. Artetxe et al. [28] show that a frozen transformer body transfers to a new language when only the embedding matrix is relearned. BLOOM+1 [29] compares continued pretraining with adapter methods such as MAD-X [30], and Zhao et al. [31] adapt LLaMA to Chinese with less than 1% of its pretraining data. For Arabic, Kuwain [9] adds about 26K tokens to TinyLlama, AraLLaMA [10] expands the vocabulary in stages, and RightNow-Arabic [11] adds 27,032 tokens to Qwen2.5-0.5B and trains on 504M Arabic tokens.

Each added token needs an initial embedding. Mean sub-token initialization averages the embeddings of the pieces produced by the old tokenizer [12]. WECHSEL [13] and FOCUS [14] use auxiliary embedding spaces to construct similarity-weighted estimates. VocADT [32] learns new rows with adapters. Hewitt [33] analyzes how added output rows change the next-token distribution; their logits relative to the original partition function matter, so a small row norm alone does not ensure a

[2]AI assistance: OpenAI Codex [3]; see Acknowledgment.

Word	Ours (Alif pres.)	Ours (Alif unif.)	AraBERT	Qwen2.5/Qwen3
الاقتصاد *definite article + derived noun*	الاقتصاد	الاقتصاد	الا + ##قت + ##صاد	ال + اقتصاد
أصعب *hamzated comparative (Alif handling)*	أصعب	اصعب	أصعب	أ + صعب
مكتباتهم *plural + possessive clitic*	مكتبات + هم	مكتبات + هم	مكتب + ##اتهم	م + كت + بات + هم
وسيكتبونها *conj + future + verb + object clitic*	وسي + كتب + ونها	وسي + كتب + ونها	وسي + ##كتب + ##ونها	و + سي + كتب + ون + ها
التكنولوجيا *long borrowing*	التكنولوجيا	التكنولوجيا	الت + ##كن + ##ولوجيا	الت + كن + ولوج + يا

FIGURE 1: Segmentation of the same Arabic words by AraToken (ours) and two other tokenizers, each after its own normalization; pieces are joined with "+". AraToken splits *their libraries* into stem and possessive clitic, where AraBERT breaks the stem and Qwen3 produces four pieces. AraToken keeps the borrowing *technology* whole, where AraBERT needs three pieces and Qwen3 four.

small change. Downey et al. [34] study the structure of the initialized matrix, and Joshi et al. [35] compare initialization choices for Hindi. Yamaguchi et al. [36] examine their dependence on the target-language budget. Tejaswi et al. [12] evaluate constituent-token averaging, whereas the Mean baseline in Mundra et al. [37] averages the original vocabulary embeddings. These are different initialization rules. Our comparison concerns a character-overlap proxy and its fixed average with the constituent estimate, not the original semantic-space WECHSEL and FOCUS methods. We state the condition under which this average reduces squared error relative to both components; its effect on language-model loss is an empirical question.

III. METHODOLOGY

AraToken reduces the number of tokens that Arabic text costs, and LEP brings its vocabulary into a pretrained model without pretraining from scratch. Figure 2 shows the whole pipeline. This section describes the normalization pipeline, the tokenizer and the three steps of LEP.[3]

A. ARABIC NORMALIZATION PIPELINE

Web Arabic writes the same word in several ways, and a tokenizer trained on raw text learns a separate piece for each spelling. The normalization pipeline removes this variation before the tokenizer sees the text. We implement it as a normalizer of the HuggingFace Tokenizers library [38], so the same rules apply during training and at inference time. The pipeline applies Unicode Normalization Form KC (NFKC), removes Tatweel (U+0640), maps Arabic-Indic digits (U+0660–U+0669) to Western digits, and maps the Arabic comma, semicolon and question mark (U+060C, U+061B, U+061F) to their Latin forms. It removes diacritics by default, and a second mode keeps them for tasks that need vocalized text. Table 1 lists the rules, and Figure 3 shows their effect on sample text.

The pipeline can also unify the four Alif variants (U+0622, U+0623, U+0625, U+0671) into bare Alif (U+0627), as many Arabic toolkits do [4], [22]. We keep the variants by default to retain these orthographic distinctions; Alif unification is an optional additional mapping. The rules apply to the whole input string, including Latin lowercasing. For example, `AI` followed by a space and Arabic-Indic digits U+0661 and U+0662 becomes `ai 12` under the released normalizer. Retaining the original token IDs therefore does not preserve case or guarantee a byte-for-byte round trip for mixed-script text.

B. TOKENIZER TRAINING

We train AraToken with the SentencePiece Unigram algorithm [18], [19] on the `tok-train` split of our corpus (Section IV-A): 159,826 documents with about 480M characters from the Arabic part of FineWeb2-HQ [39]. We chose Unigram after comparing it with BPE and

[3]AI assistance: OpenAI Codex [3]; see Acknowledgment.

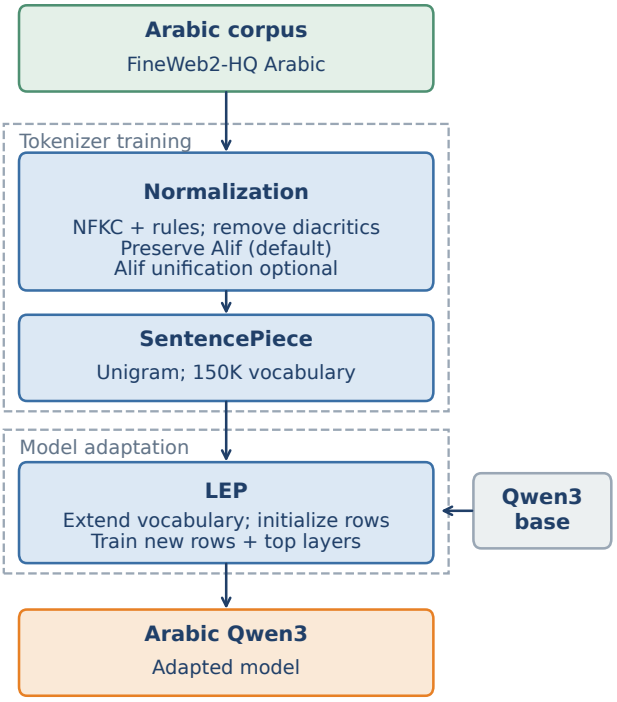


FIGURE 2: Overview of AraToken and LEP (ours). Arabic text passes through the normalization pipeline, a SentencePiece Unigram tokenizer is trained on the normalized corpus, and LEP adds its vocabulary to Qwen3. NFKC denotes Unicode Normalization Form KC.

Category	Original	Normalized
Alif Variants	أ (U+0623)	ا (U+0627)
	إ (U+0625)	ا (U+0627)
	آ (U+0622)	ا (U+0627)
	ٱ (U+0671)	ا (U+0627)
Numerals	٠–٩	0–9
	٫ (U+066B)	.
	٬ (U+066C)	,
Punctuation	؟ (U+061F)	?
	؛ (U+061B)	;
	، (U+060C)	,
Tatweel	ـ (U+0640)	(removed)

TABLE 1: Character rules of the AraToken normalization pipeline (ours). Alif unification is available as an option and is off by default.

Original	Normalized	Transformation
أَكَلَ	اكل	Optional Alif unification + diacritics removed
إِسْلَام	اسلام	Optional Alif unification + diacritics removed
آلاف	الاف	Optional Alif unification
٠١٢٣	0123	Arabic-Indic to Western numerals
الـكـتـاب	الكتاب	Tatweel removed

FIGURE 3: Sample text before and after AraToken normalization (ours). The first three rows illustrate optional Alif unification; the first two also remove diacritics.

WordPiece under every preprocessing configuration (Section V-B). The target vocabulary has 150,000 pieces, close to the 151,643 regular tokens of Qwen3. We also evaluate a separate SentencePiece variant with byte fallback, which represents otherwise unsupported characters using the 8-bit Unicode Transformation Format (UTF-8). The extended Qwen tokenizer retains its base byte-level coverage; the standalone Unigram and byte-fallback variants are distinguished in the measurements.

A 150K vocabulary adds many embedding rows. For tighter memory budgets we prune the vocabulary by corpus coverage: we keep the most frequent pieces that together cover 99% or 95% of the tokens of the training corpus. This gives vocabularies of 109,844 and 60,149 pieces (Section V-C).

C. LANGUAGE EXTENSION PIPELINE

LEP adds the AraToken vocabulary to Qwen3 in three steps. It extends the vocabulary, initializes the new embedding rows, and trains the new rows together with the top transformer layers (Figure 4).

1) Vocabulary extension

Let $\mathcal{T}_{\text{ar}}$ be AraToken and $\mathcal{T}_{\text{qwen}}$ the Qwen3 tokenizer. We take every piece of $\mathcal{T}_{\text{ar}}$ that is not in $\mathcal{T}_{\text{qwen}}$ and drop the pieces that contain Latin or Cyrillic letters, bare digits or punctuation, which Qwen3 already encodes well. The Qwen3-0.6B tokenizer has 151,669 token IDs, whereas its embedding matrix has 151,936 rows. The extended tokenizer has 282,826 IDs: 131,157 new IDs reuse 267 padded positions and require 130,890 appended rows. In the initialization formulas, n denotes the original tokenizer size, M the embedding matrix, and $d = 1024$ the embedding dimension. Qwen3-0.6B ties its input and output embeddings, so the appended rows add 134M physical parameters once, and the model grows from 596M to 730M.

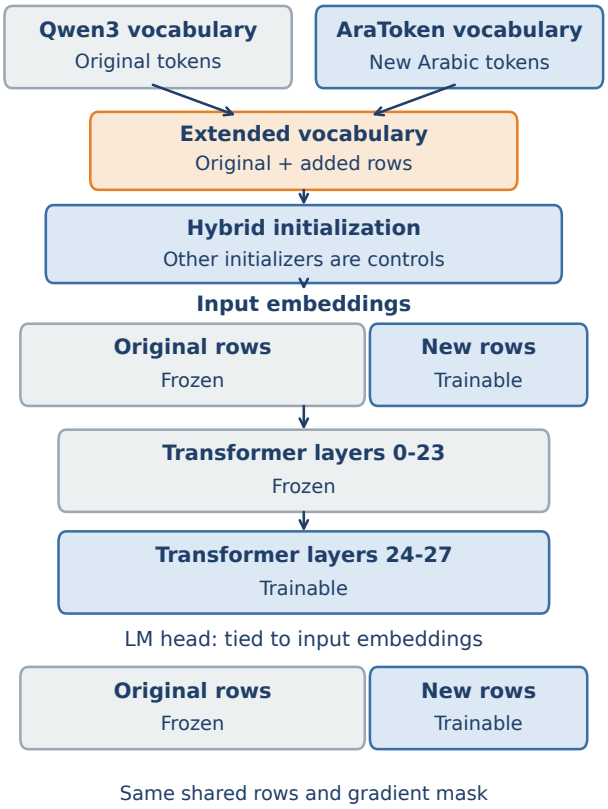


FIGURE 4: The Language Extension Pipeline (ours). New Arabic tokens are appended to the Qwen3 vocabulary and their embedding rows are initialized. During training, gradient masking keeps the original rows fixed, and only the new rows and the last four transformer layers are updated. LM denotes language model.

2) Initialization of the new rows

Each new token t_k needs an initial row $\boldsymbol{M}_{n+k}$. LEP computes two estimates of this row and takes their average.

The compositional estimate encodes the surface form of t_k with $\mathcal{T}_{\text{qwen}}$, which gives a sequence S_k of sub-token occurrences, including repetitions, and averages their embeddings:

$$\boldsymbol{c}_k = \frac{1}{|S_k|} \sum_{s \in S_k} \boldsymbol{M}_s. \tag{1}$$

Proposition 1 describes where this estimate lies.

Proposition 1: Let the sub-token embeddings $\boldsymbol{M}_s$, $s \in S_k$, have mean $\boldsymbol{\mu}$, $\mathbb{E}\|\boldsymbol{M}_s - \boldsymbol{\mu}\|^2 = r^2$, and $\mathbb{E}\langle \boldsymbol{M}_s - \boldsymbol{\mu}, \boldsymbol{M}_t - \boldsymbol{\mu}\rangle = \rho r^2$ for $s \neq t$. Then

$$\mathbb{E}\|\boldsymbol{c}_k - \boldsymbol{\mu}\|^2 = \frac{1 + (|S_k| - 1)\rho}{|S_k|}\, r^2. \tag{2}$$

Proof. Write $\boldsymbol{c}_k - \boldsymbol{\mu}$ as the mean of the $|S_k|$ centered vectors and expand the squared norm. The $|S_k|$ diagonal terms contribute $|S_k| r^2$ and the $|S_k|(|S_k|-1)$ cross terms contribute ρr^2 each. Dividing by $|S_k|^2$ gives the result. □

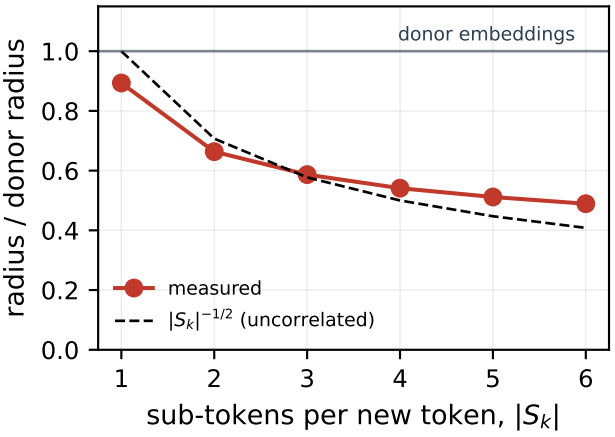


FIGURE 5: Radius of the compositional estimate $\boldsymbol{c}_k$ relative to the donor embedding radius, by the number of Qwen3 sub-tokens of the new token, for 20,000 candidate rows. Points are arithmetic mean centered radii; the dashed line is the idealized root-mean-square reference $|S_k|^{-1/2}$ from Proposition 1.

Under the stated moment assumptions and $\rho = 0$, the root-mean-square centered radius scales as $|S_k|^{-1/2}$. Figure 5 reports a historical diagnostic on 20,000 newly appended rows, selected by increasing row ID. The mean radius is 0.60 of the mean donor radius, falling from 0.89 for single-piece tokens to 0.49 for six-piece tokens. These arithmetic mean radii and conditional donor groups do not directly measure the proposition's second moment or identify ρ.

With tied embeddings, a new row also sets the output logit $\boldsymbol{h}^\top \boldsymbol{M}_{n+k}$ of its token. For an idealized extension at a fixed hidden state, with all old logits unchanged, the next-token distributions satisfy [33]

$$\mathrm{KL}\left(p_{\text{old}} \,\|\, p_{\text{new}}\right) = \log\left(1 + \frac{1}{Z} \sum_k \exp\left(\boldsymbol{h}^\top \boldsymbol{M}_{n+k}\right)\right), \tag{3}$$

Here KL denotes Kullback–Leibler divergence, $\boldsymbol{h}$ is the hidden state and Z is the partition function over the original vocabulary. Small row norms alone do not ensure small divergence: if K rows are added, zero initialization still contributes K to the added partition mass. Centered radius also differs from absolute norm. This identity does not model input retokenization, changes to pre-existing rows or reuse of padded output rows. We retain the unscaled compositional estimate as an initialization choice whose effect requires empirical evaluation, rather than as a distribution-preservation guarantee.

Our analogical estimate uses character-overlap similarity to tokens of the old vocabulary. This is a lexical proxy, not an implementation of the learned semantic auxiliary

spaces of WECHSEL [13] or FOCUS [14]. Donor tokens are converted to surface strings by the base tokenizer. We strip surrounding whitespace and count overlapping character 2–4-grams, using the whole stripped string as one feature if it has no such gram. Let P_g contain the first 400 donor IDs having gram g, in ascending ID order, and let $q_k(g)$ be its target count. Candidate s receives score $b_{ks} = \sum_g q_k(g)\mathbf{1}\{s \in P_g\}$. The set $\mathcal{N}_k$ contains the ten highest positive scores, with $w_s = b_{ks}/\sum_{j\in\mathcal{N}_k} b_{kj}$. Ties follow first encounter when scanning grams by increasing length and position, then postings by ID. With no candidate, we use the compositional estimate; if composition is also unavailable, we use the random control defined below. With candidates but no composition, the hybrid uses the analogical estimate alone. Then

$$\boldsymbol{a}_k = \sum_{s\in\mathcal{N}_k} w_s \boldsymbol{M}_s. \tag{4}$$

The estimates use different surface information: the base tokenizer's decomposition and character overlap with donor tokens. To test whether combining them helps, LEP uses their average,

$$\boldsymbol{M}_{n+k} = \tfrac{1}{2}\left(\boldsymbol{c}_k + \boldsymbol{a}_k\right), \tag{5}$$

and Proposition 2 gives its squared error relative to a fixed reference row.

Proposition 2: Let $\boldsymbol{v}_k$ be the ideal row of t_k, and let $\boldsymbol{\varepsilon}_{\mathrm{c}} = \boldsymbol{c}_k - \boldsymbol{v}_k$ and $\boldsymbol{\varepsilon}_{\mathrm{a}} = \boldsymbol{a}_k - \boldsymbol{v}_k$ be the errors of the two estimates. The error of the hybrid (5) is

$$\|\boldsymbol{M}_{n+k} - \boldsymbol{v}_k\|^2 = \tfrac{1}{2}\left(\|\boldsymbol{\varepsilon}_{\mathrm{c}}\|^2 + \|\boldsymbol{\varepsilon}_{\mathrm{a}}\|^2\right) - \tfrac{1}{4}\|\boldsymbol{c}_k - \boldsymbol{a}_k\|^2. \tag{6}$$

The hybrid error is therefore never larger than the mean of the two errors, and it is smaller than the smaller of them whenever $\|\boldsymbol{c}_k - \boldsymbol{a}_k\|^2 > 2\left|\|\boldsymbol{\varepsilon}_{\mathrm{c}}\|^2 - \|\boldsymbol{\varepsilon}_{\mathrm{a}}\|^2\right|$.

Proof. Apply the parallelogram identity $\|\boldsymbol{x}+\boldsymbol{y}\|^2 + \|\boldsymbol{x}-\boldsymbol{y}\|^2 = 2\|\boldsymbol{x}\|^2 + 2\|\boldsymbol{y}\|^2$ to $\boldsymbol{x} = \boldsymbol{\varepsilon}_{\mathrm{c}}$ and $\boldsymbol{y} = \boldsymbol{\varepsilon}_{\mathrm{a}}$, divide by four, and note that $\boldsymbol{\varepsilon}_{\mathrm{c}} - \boldsymbol{\varepsilon}_{\mathrm{a}} = \boldsymbol{c}_k - \boldsymbol{a}_k$. □

The identity bounds error against the mean component error; beating both components requires the stronger condition stated above. Neither the ideal row nor its component errors is observed, and Euclidean embedding error does not by itself bound language-model loss. We therefore evaluate the components and mixture empirically. The primary weight remains one half; the mixture needs no data beyond the two component constructions.

3) Gradient masking and selective unfreezing

During adaptation, LEP keeps the original embedding rows fixed. A backward hook sets their gradients to zero:

$$\nabla_{\boldsymbol{M}_i}\mathcal{L} = \begin{cases} 0 & \text{if } i < n, \\ \nabla_{\boldsymbol{M}_i}\mathcal{L} & \text{otherwise.} \end{cases} \tag{7}$$

The masked embedding table is excluded from AdamW weight decay so that zero-gradient rows remain fixed. Because the embeddings are tied, the hook also fixes the output rows of the original vocabulary. LEP also adapts the top transformer layers to the new segmentation: layers 24–27 of the 28-layer Qwen3 models. In the primary 0.6B policy, the 131,157 newly assigned vocabulary rows and four blocks account for 197,228,544 effectively updated parameters; the rest stays frozen. Fully trainable controls are specified below, and historical layer-selection diagnostics appear in Appendix D.

IV. EXPERIMENTAL SETUP

The new adaptation study tests whether hybrid initialization changes Arabic language-modeling quality relative to its components and control initializers, and how the resulting recipe compares with continued pretraining (CPT) at the same nominal token budget. It also measures English likelihood and selected task accuracies after adaptation. These are separate endpoints; improved Arabic likelihood does not establish task transfer or English retention. Historical tokenizer and adaptation diagnostics are reported separately. Below we specify the data, models, training policies, metrics and statistical analysis.[4]

A. DATA

Tokenizer training and Arabic adaptation use the released `arb_Arab` subset of FineWeb2-HQ [39]. The split manifest records a minimum document length of 200 characters and no additional positive quality-score cutoff. Corpus IDs are assigned deterministically to tokenizer-training, language-model-training, development and test buckets. The stored splits contain 159,826, 662,685, 2,522 and 6,602 documents, respectively. Development data monitor training; the final-step checkpoint is selected independently of development or test scores. Different IDs may still contain overlapping text. Appendix A gives the split and overlap-diagnostic details.

The new study reuses these previously inspected data and the released packed training streams; this follow-up therefore does not evaluate a new untouched holdout. English likelihood and downstream tasks use the separate datasets specified below.

B. MODELS AND BASELINES

We use Qwen3-0.6B-Base and Qwen3-1.7B-Base [15], each with 28 transformer blocks and tied input and output embeddings; their hidden dimensions are 1,024 and 2,048. The released extended tokenizer is fixed before the new runs. Our primary LEP policy updates its newly assigned vocabulary rows and blocks 24–27 while preserving the original-token rows. CPT retains the base tokenizer and updates its embedding table and the same four blocks. Both policies freeze the final normalization parameters. Additional controls train every parameter of the extended model or base model, including the original

[4]AI assistance: OpenAI Codex [3]; see Acknowledgment.

Setting	Steps	Arms	Runs
0.6B, original budget	2,000	10	30
0.6B, nominal 500M	5,087	8	24
1.7B, original budget	2,000	8	24

TABLE 2: The frozen 78-run adaptation design for LEP (ours) and CPT. Each setting contains five initializer arms, top-four-layer CPT and two fully trainable controls; the smaller 0.6B setting adds two mixture weights. Every arm has three training seeds. Tokenizer/model size, row policies and text exposure differ between LEP and CPT.

embedding rows and final normalization. These fully trainable controls differ from the historical masked, all-layer LEP diagnostic.

For each setting, the initializer comparison includes random, constituent sub-token mean, moment-matched mean, character-n-gram analogical transfer and the equal-weight hybrid. The analogical component uses the implemented lexical proxy, not the pretrained auxiliary semantic embeddings of WECHSEL or FOCUS. At the smaller 0.6B budget, two additional mixtures give $\alpha \in \{0, .25, .5, .75, 1\}$ together with the component endpoints. The primary $\alpha = .5$ is fixed; the sensitivity analysis does not select a replacement. Every registered arm uses training seeds 1, 2 and 3. Table 2 gives the complete design.

The random control draws $\mathcal{N}(0, \sigma^2 I_d)$, where I_d is the identity matrix for the backbone hidden dimension d and σ is the sample standard deviation across all entries of the original-token rows, excluding padding. The moment-matched control uses donor mean $\boldsymbol{\mu}$ and centered radii $r_i = \|\boldsymbol{M}_i - \boldsymbol{\mu}\|$. For $\boldsymbol{u}_k = \boldsymbol{c}_k - \boldsymbol{\mu}$ and a uniformly sampled donor row J, it assigns $\boldsymbol{\mu} + r_J \boldsymbol{u}_k / \|\boldsymbol{u}_k\|$ when $\|\boldsymbol{u}_k\| \geq 10^{-6}$. Otherwise it draws $\boldsymbol{z} \sim \mathcal{N}(0, I_d)$ before sampling J and assigns $\boldsymbol{\mu} + r_J \boldsymbol{z}/\sqrt{d}$. The regular branch matches sampled centered radii, not full covariance; the near-zero branch matches the expected squared radius. Empty base-tokenization results use the random fallback; the ordinary mean arm retains the library resize initialization. Initialization uses the recorded seed and float32 arithmetic before conversion to bfloat16 (BF16).

C. TRAINING

The new runs use AdamW with learning rate 2×10^{-4}, linear decay, 10% warmup and weight decay 0.01; embedding tables subject to frozen-row masking are excluded from decay. The global batch is nominally 96 packed sequences of 1,024 tokens. Runs last either 2,000 steps (196,608,000 nominal input tokens) or 5,087 steps (500,072,448 nominal input tokens). All settings use three training seeds. We evaluate the final completed step, with development-based checkpoint selection disabled. Actual input-token counts, successful-run wall time and peak allocated/reserved graphics processing unit (GPU) memory are recorded separately from nominal budgets; input tokens are not the number of unmasked prediction targets.

Equal nominal token budgets do not equalize consumed characters, model size, floating-point operations (FLOPs) or wall time. The packed streams use the respective tokenizers and have different lengths, so exposure and repetition can differ. Initialization and unsuccessful infrastructure attempts are accounted for separately. No run, seed or checkpoint is selected for favorable evaluation performance.

D. METRICS

Intrinsic comparisons use Qwen as the unextended backbone control, DeepSeek-V3 as an additional multilingual BPE tokenizer and AraBERT [7] as an Arabic WordPiece baseline. The historical tokenizer evaluation reports fertility (tokens per whitespace-delimited word) and compression (Unicode characters per token) as per-document means, with corpus aggregates identified separately. The main intrinsic artifact uses 5,000 test documents; the 80K algorithm/preprocessing comparison uses 2,000. The baseline table gives raw and common-normalized inputs, and its figure uses common-normalized inputs. Algorithm and pruning comparisons use each tokenizer's saved normalization. Word-level out-of-vocabulary (OOV) rates and unknown-token (UNK) fractions accompany compression comparisons, since an unknown token can conceal unsupported input; unscored rates are marked separately. Tokens per morpheme is a corpus ratio over the first 500 test documents: total tokenizer tokens divided by total CAMeL `d3tok` segments [22]. CAMeL receives common normalized text, while each tokenizer receives its declared own-normalized input. These diagnostics do not constitute additional training seeds in the new study.

Language-model evaluation uses a common-text protocol. For each document, the alif-preserving paper normalizer is applied to a fixed point before taking at most 512 Unicode code points. The normalizer includes Latin lowercasing, so the English endpoint is normalized-text likelihood. The same canonical span is scored by every compared model, without tokenizer-specific truncation. An explicit common conditioning token (ID 151643) precedes the target; every target token contributes to the summed negative log likelihood, while the conditioning token is excluded. Let c be the canonical character span, $(t_1, \ldots, t_m)$ its target-token sequence and t_0 the common conditioning token. Negative log likelihood (NLL) in nats is $\mathrm{NLL}(t_i) = -\ln p(t_i \mid t_0, \ldots, t_{i-1})$. We compute

$$\mathrm{BPC}(c) = \frac{\sum_i \mathrm{NLL}(t_i)}{|c| \ln 2}. \tag{8}$$

The primary aggregate is the mean of document BPC values; corpus BPC divides total negative log likelihood by the total character count. The Arabic cohort is the first 1,500 eligible documents in the stored test order, and the English cohort is the first 1,000 WikiText-103

[40] raw test rows of at least 200 raw characters. Source hashes, canonical hashes, character counts and evaluation protocols must match across comparisons. Historical values obtained from independently truncated 1,024-token spans remain separate and are not used as common-text comparisons.

The fixed downstream panel evaluates seven logical checkpoints: one pinned 0.6B base and all three training seeds of the nominal-500M hybrid and top-four-layer CPT arms. Five Arabic tasks from the Open Arabic LLM Leaderboard (OALL) v2 [41] (ArabicMMLU [42], MadinahQA, Arabic EXAMS [43], AraTrust [44] and AlGhafa [45]) and two English tasks (MMLU [46] and ARC-Easy [47]) give 49 checkpoint–task blocks. The Arabic endpoint is unweighted mean accuracy across the five fixed tasks; pooled accuracy and every task score are also reported. The English task endpoint is the unweighted mean of its two tasks. These are custom-protocol subset measurements, not official leaderboard submissions. Task accuracy at 1.7B or under the fully trainable controls is outside this panel.

We report measured training time and peak memory separately from tokenizer compression, and use a fixed serving workload to measure tokenization, prefill, decode, generation and end-to-end timing, generated-token/character throughput and peak memory at their recorded measurement scopes.

E. STATISTICAL ANALYSIS

For the new study, individual training-seed outcomes and their mean and sample standard deviation are retained. Arabic BPC inference uses a crossed paired bootstrap [48] over the three training seeds and common documents, with 10,000 replicates and resampling seed 42. We report marginal 95% percentile intervals and two-sided, centered-bootstrap Monte Carlo p-values with the plus-one correction. Holm correction [49] is applied separately to the six predefined LEP–CPT contrasts (two parameter policies at each of three settings) and the twelve hybrid–initializer contrasts (four controls at each setting). An incomplete family receives no inferential conclusion. The original-budget mixture-weight sensitivity is exploratory and does not select the primary initializer.

The downstream family contains nine contrasts: hybrid–CPT, hybrid–base and CPT–base for Arabic macro task accuracy, English macro task accuracy and English macro document BPC. The task bootstrap draws training seeds jointly across the fixed tasks and resamples paired items within each task; tasks themselves are not resampled. The fixed base is repeated as a reference, not treated as three independently trained models. Inference and Holm-adjusted p-values are withheld until the complete nine-contrast family validates. All per-task and per-seed values remain visible. No equivalence or noninferiority margin is defined, so nonsignificance does not establish preserved performance.

V. RESULTS

We first examine tokenization and pruning, then controlled adaptation and initialization, followed by resource use and Arabic task accuracy with English retention.[5]

A. ARATOKEN AGAINST EXISTING TOKENIZERS

Table 3 and Figure 6 compare our AraToken with the tokenizers of Qwen2.5/Qwen3, DeepSeek-V3 and AraBERT. AraToken gives the lowest fertility and the highest compression of all systems. On normalized text it needs 1.212 tokens per word, against 2.179 for Qwen2.5/Qwen3, 2.177 for DeepSeek-V3 and 1.833 for AraBERT. This is 44.4% fewer tokens than the tokenizer of Qwen3 and 33.9% fewer than AraBERT, and each token covers 4.80 characters, against 2.67 and 3.17. AraToken also leads on raw, unnormalized text, where it needs 40.1% fewer tokens than the tokenizer of Qwen3. Raw-input results still include each tokenizer's internal preprocessing and do not isolate segmentation alone.

Tokenizer	Vocab	Fertility ↓ raw	Fertility ↓ norm.	Compr. ↑ norm.
Qwen2.5	151,665	2.283	2.179	2.669
DeepSeek-V3	128,815	2.238	2.177	2.670
AraBERT	64,000	1.796	1.833	3.170
Ours, pruned				
AraToken, 95% coverage	60,149	1.431	1.281	4.549
AraToken, 99% coverage	109,844	1.380	1.227	4.745
Ours, full scale				
AraToken, 150K	150,000	**1.366**	**1.212**	**4.802**
AraToken, 150K, byte fallback	150,000	1.512	1.215	4.792

TABLE 3: AraToken (ours) against existing tokenizers on 5,000 `test` documents. Fertility is tokens per word and compression is characters per token, both as per-document means. Every tokenizer is measured on raw text and on common text normalized by the AraToken pipeline. Qwen3 has 151,669 token IDs and the same displayed scores as Qwen2.5. The best values are in bold.

Part of the gap to AraBERT could come from vocabulary size, since AraToken has 150K pieces and AraBERT 64K. Our pruned AraToken with 95% coverage has 60,149 pieces, fewer than AraBERT, and it still needs 30.1% fewer tokens per word. The advantage therefore persists at a similar vocabulary size. This comparison does not isolate vocabulary size from segmentation, training data or normalization. AraToken also needs the fewest tokens per morpheme: 0.89, against 1.32 for AraBERT and 1.57 for Qwen3, with morphemes from the CAMeL Tools analyzer. This corpus ratio describes granularity relative to the analyzer's reference; it is not a measure of morpheme-boundary accuracy. On dialectal tweets from ArSarcasm, a register far from its training corpus, our AraToken byte-fallback variant still needs 21.8%

[5] AI assistance: OpenAI Codex [3]; see Acknowledgment.

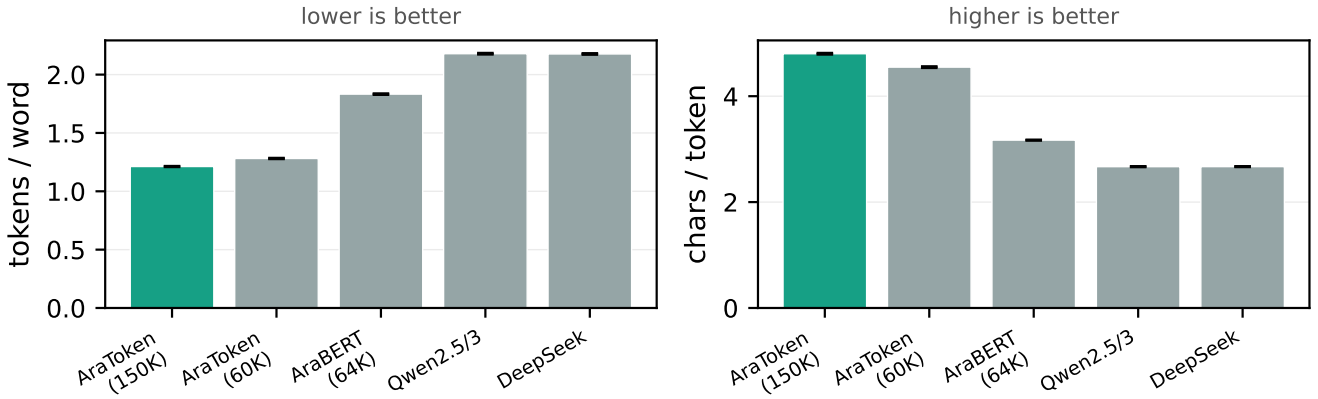


FIGURE 6: Fertility (left, lower is better) and compression (right, higher is better) of AraToken (ours) and existing tokenizers on 5,000 test documents with common externally normalized input and 95% bootstrap intervals.

fewer tokens per word than the tokenizer of Qwen3 (Appendix B).

B. CHOICE OF SEGMENTATION ALGORITHM

Table 4 compares BPE, WordPiece and Unigram, trained on the same corpus at an 80K vocabulary, with and without the steps of the normalization pipeline. Unigram gives the lowest fertility and the highest compression in every configuration. Our full-pipeline Unigram configuration needs 1.245 tokens per word, against 1.295 for BPE and 1.296 for WordPiece, and it covers 4.69 characters per token, against 4.51 for both. Its rate of unknown words is 0.28% in this 80K comparison. A separate 150K byte-fallback variant is evaluated in Appendix B.

Both steps of our normalization pipeline lower fertility for all three algorithms (Figure 7). Removing diacritics gives the larger gain: 7.0% for BPE, 7.1% for WordPiece and 2.6% for Unigram. The character rules add about 0.6% for each algorithm, and the full pipeline lowers fertility by 7.5%, 7.6% and 3.2%.

Tokenizer	Fertility ↓	Compression ↑	OOV% ↓
Diacritics removed			
BPE	1.303	4.49	–
BPE + char. rules	1.295	4.51	–
Unigram	1.253	4.66	0.14
Unigram + char. rules	1.245	4.69	0.28
WordPiece	1.304	4.48	0.00
WordPiece + char. rules	1.296	4.51	0.00
Diacritics kept			
BPE	1.400	4.33	–
BPE + char. rules	1.392	4.35	–
Unigram	1.286	4.62	0.15
Unigram + char. rules	1.278	4.64	0.28
WordPiece	1.402	4.32	0.00
WordPiece + char. rules	1.395	4.34	0.00

TABLE 4: Segmentation algorithms at an 80K vocabulary, trained on `tok-train` and measured on 2,000 `test` documents under each tokenizer's own normalization. "Char. rules" adds the character rules of Table 1. Unigram with the full AraToken pipeline (ours) (bold) gives the lowest fertility and the highest compression. Out-of-vocabulary (OOV) rate is scored on the first 500 documents, with 224,412–224,503 words depending on normalization. "–" means not scored (BPE has no usable unknown-token (UNK) marker).

C. VOCABULARY PRUNING

Table 5 and Figure 8 show the cost of pruning. At 99% coverage the vocabulary shrinks by 27%, to 109,844 pieces, and fertility rises by 1.2%, from 1.212 to 1.227. At 95% coverage it shrinks by 60%, to 60,149 pieces, for a rise of 5.7% to 1.281. Both of our pruned tokenizers stay well ahead of every baseline in Table 3. Pruning removes the long and rare pieces of the vocabulary (Appendix C), so one coverage threshold sets the trade-off between vocabulary size and fertility.

Configuration	Vocab	Fertility ↓	Compr. ↑
full vocabulary	150,000	1.212	4.80
99% coverage	109,844	1.227	4.74
95% coverage	60,149	1.281	4.55

TABLE 5: AraToken (ours) pruned by corpus coverage, measured on the first 5,000 `test` documents under each tokenizer's own normalization.

D. ADAPTATION QUALITY

Tables 6 and 7 report all six controlled LEP–CPT comparisons. Every arm contains three completed training seeds evaluated on the same 1,500 canonical Arabic spans. LEP uses the fixed half-weight initializer. Top-four-layer comparisons allow embedding adaptation in both methods; the additional fully trainable pairs update every parameter. The comparison is at matched nominal packed-token budgets, not equal compute or equal

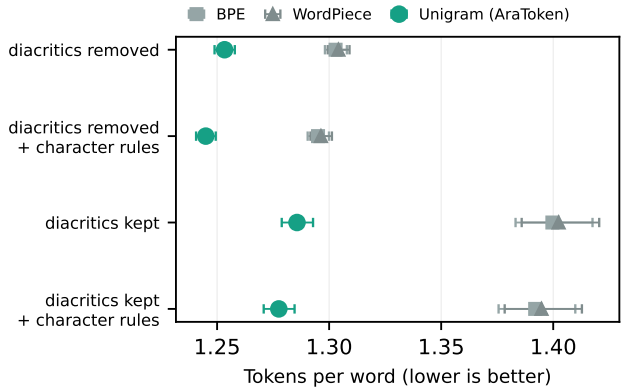


FIGURE 7: Fertility under AraToken normalization ablations (ours), evaluated with three segmentation algorithms on the `test` split under four preprocessing configurations: 2,000 documents with each tokenizer's saved normalization and 95% bootstrap intervals.

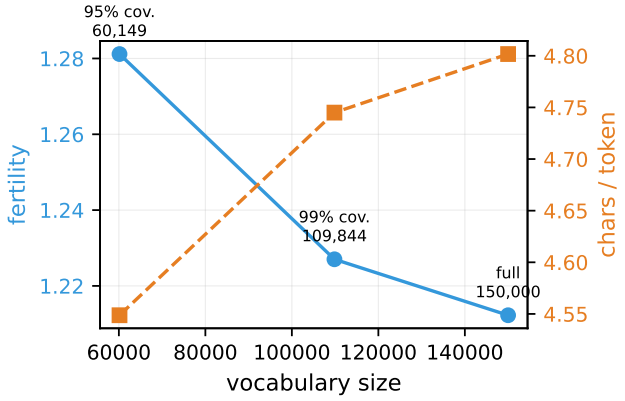


FIGURE 8: Vocabulary size and fertility of AraToken (ours) at full size and at 99% and 95% coverage.

character exposure.

With the top-four policy, our LEP reaches 1.4111 versus 1.5138 BPC at 2,000 steps, 1.3551 versus 1.4782 at the nominal-500M budget, and 1.3224 versus 1.3577 at 1.7B. The paired LEP–CPT differences are −0.1027, −0.1231 and −0.0353 BPC; their marginal 95% intervals are $[-0.1066, -0.0989]$, $[-0.1277, -0.1187]$ and $[-0.0395, -0.0313]$. The fully trainable controls retain advantages of 0.0986, 0.1029 and 0.0611 BPC in the same order. All six differences survive the declared Holm correction ($p_{\text{Holm}} = 0.000600$).

These controls support a canonical-Arabic likelihood benefit under both trainability policies and at both base-model sizes. They do not isolate segmentation from normalization, character exposure or the additional embedding capacity. The resource comparison below records the higher training cost, and the task and English measurements test separate outcomes. Individual seeds and all declared initializer arms are retained in the accompanying result files.

E. INITIALIZATION OF THE NEW ROWS

Tables 8 and 10 compare our fixed half-weight mixture with compositional mean, moment-matched mean, character-ngram analogical transfer and random initialization under all three scale/budget settings. These comparisons hold the extended tokenizer, trainable policy and schedule fixed within each setting. The character-ngram method is a lexical proxy, not a reimplementation of WECHSEL or FOCUS.

Against compositional initialization, our mixture lowers Arabic BPC by 0.0485 at the original 0.6B budget, 0.0402 at the nominal-500M budget and 0.0439 at 1.7B. Its advantages over analogical initialization are smaller: 0.0063 at the original budget and 0.0117 at 1.7B, with marginal 95% intervals for mixture minus analogical of $[-0.0080, -0.0043]$ and $[-0.0135, -0.0097]$. These comparisons survive Holm correction over the 12 declared initializer contrasts ($p_{\text{Holm}} = 0.0012$). At the nominal-500M budget, the difference from analogical initialization is unresolved: −0.0009 BPC, interval $[-0.0028, 0.0010]$, $p_{\text{Holm}} = 0.383$; one of the three seed differences favors the analogical component. Random and moment-matched initialization have higher Arabic BPC in all three settings.

The mixture therefore improves on both components in two of the three settings, while its advantage over the stronger component narrows at the larger 0.6B budget. This is a budget-dependent empirical result, not a consequence of the squared-error identity or evidence that either component fails in isolation. With only three training seeds, the crossed-bootstrap intervals have limited coverage of training variability.

In the exploratory original-budget sweep, where α is the compositional weight, Arabic BPC is 1.4174, 1.4052, 1.4111, 1.4289 and 1.4596 at $\alpha = 0, 0.25, 0.5, 0.75, 1$, respectively. The observed minimum is at 0.25. We retain the predeclared half-weight mixture as the primary method; these results do not establish that equal weighting is optimal. Table 9 reports English likelihood for every initializer; Table 11 gives the complete sensitivity sweep. The endpoint at $\alpha = 1$ is represented by the declared compositional-only arm, not an additional trained mixture. All individual seed values are retained.

F. COMPRESSION, TRAINING COST AND SERVING

The released packed adaptation streams contain 448,343,040 input tokens with the extended tokenizer and 802,561,024 with the base tokenizer. Fewer tokens therefore represent the same source corpus, subject to the packing remainders described in Appendix F. This is an input-compression result, not a generation-throughput measurement. At 2,000 steps both methods consume 196,608,000 packed input tokens; their character exposure and FLOPs differ. At 5,087 steps, actual counts are

Model	Steps	Trainability	CPT Arabic ↓	LEP Arabic ↓	CPT English ↓	LEP English ↓
0.6B	2,000	Top four	1.5138 ± 0.0005	1.4111 ± 0.0003	1.2249 ± 0.0014	1.2080 ± 0.0006
0.6B	2,000	Full	1.3780 ± 0.0009	1.2794 ± 0.0039	1.3407 ± 0.0103	1.2945 ± 0.0070
0.6B	5,087	Top four	1.4782 ± 0.0004	1.3551 ± 0.0017	1.2430 ± 0.0014	1.2159 ± 0.0008
0.6B	5,087	Full	1.3226 ± 0.0022	1.2197 ± 0.0006	1.3669 ± 0.0030	1.3090 ± 0.0025
1.7B	2,000	Top four	1.3577 ± 0.0010	1.3224 ± 0.0029	1.0842 ± 0.0033	1.0721 ± 0.0011
1.7B	2,000	Full	1.2699 ± 0.0009	1.2088 ± 0.0010	1.1207 ± 0.0024	1.1183 ± 0.0039

TABLE 6: Arabic macro document BPC on 1,500 shared canonical documents, each truncated to at most 512 Unicode characters. Lower BPC is better. Values are means ± sample standard deviation (SD) across three paired training seeds. English uses 1,000 canonical documents with the same prefix rule and is descriptive. Top-four LEP updates new embedding rows and the last four transformer layers; CPT updates all original embedding rows and the same layers. Full pairs update all parameters. LEP (ours) uses the predeclared half mixture. Matched nominal packed-token budgets do not imply equal compute or text exposure. The held-out data had previously been inspected.

Model	Steps	Trainability	Δ BPC ↓	95% interval	Holm p
0.6B	2,000	Top four	-0.1027	[-0.1066, -0.0989]	0.00059994
0.6B	2,000	Full	-0.0986	[-0.1033, -0.0937]	0.00059994
0.6B	5,087	Top four	-0.1231	[-0.1277, -0.1187]	0.00059994
0.6B	5,087	Full	-0.1029	[-0.1077, -0.0987]	0.00059994
1.7B	2,000	Top four	-0.0353	[-0.0395, -0.0313]	0.00059994
1.7B	2,000	Full	-0.0611	[-0.0646, -0.0577]	0.00059994

TABLE 7: Arabic macro document BPC on 1,500 shared canonical documents, each truncated to at most 512 Unicode characters. Lower BPC is better. LEP (ours) minus CPT for all six trainability comparisons. Negative differences favor LEP. Intervals are 95% marginal crossed seed/document bootstrap intervals over three paired training seeds and 1,500 shared documents; they are not simultaneous intervals. Holm correction covers the six planned fairness contrasts. The held-out data had previously been inspected.

Initializer	0.6B / 2,000 steps BPC ↓	0.6B / 5,087 steps BPC ↓	1.7B / 2,000 steps BPC ↓
Compositional mean	1.4596 ± 0.0018	1.3953 ± 0.0007	1.3664 ± 0.0042
Moment-matched mean	1.5362 ± 0.0058	1.4534 ± 0.0043	1.4698 ± 0.0085
Character-ngram analogical proxy	1.4174 ± 0.0014	1.3560 ± 0.0012	1.3341 ± 0.0025
Half mixture (ours)	1.4111 ± 0.0003	1.3551 ± 0.0017	1.3224 ± 0.0029
Random	2.3270 ± 0.0082	2.0906 ± 0.0118	2.1965 ± 0.0131

TABLE 8: Arabic macro document BPC on 1,500 shared canonical documents, each truncated to at most 512 Unicode characters. Lower BPC is better. Values are means ± sample standard deviation (SD) across three paired training seeds. All five predeclared initializers use the top-four training policy. The analogical initializer is a character-ngram proxy. No initializer is selected from these results. The held-out data had previously been inspected.

500,097,024 for our LEP and 500,072,448 for CPT because of a partial accumulation group at the extended stream's epoch boundary.

Measured training cost is higher for our LEP. For the top-four-layer pairs, mean trainer runtime is 1.25 times CPT at each 0.6B budget and 1.28 times CPT at 1.7B; peak allocated memory ratios are 1.77, 1.77 and 1.67, respectively. Fully trainable LEP takes 1.23, 1.23 and 1.25 times the corresponding CPT runtime. These are measured runtime ratios on the recorded H100 workers, not controlled estimates of FLOPs. Across all 78 successful training runs, summed trainer runtime is 272.57 single-GPU hours. Initialization, evaluation and infrastructure failures are accounted for separately; incomplete timing records for the quarantined launch prevent an exact total campaign-cost claim. Table 12 gives absolute training costs.

Table 13 reports all six fixed-checkpoint serving measurements. At 0.6B, our LEP and CPT have similar observed token rates: 61.61 versus 61.53 tokens/s at batch one and 227.09 versus 228.35 at batch four. At 1.7B, LEP is slower: 55.67 versus 56.95 and 206.99 versus 208.78 tokens/s. Generated-character throughput is 1.51–1.65 times CPT across these settings, while allocated memory is higher. The generated strings differ and can become repetitive, so the character-rate gain is not a quality-adjusted speedup or a reduction in equal-output latency. The table and raw records separate end-to-end batch time, generation throughput and memory; no serving significance claim is made.

Initializer	0.6B / 2,000 steps BPC ↓	0.6B / 5,087 steps BPC ↓	1.7B / 2,000 steps BPC ↓
Compositional mean	1.2087 ± 0.0008	1.2172 ± 0.0009	1.0705 ± 0.0008
Moment-matched mean	1.2111 ± 0.0005	1.2212 ± 0.0008	1.0740 ± 0.0019
Character-ngram analogical proxy	1.2069 ± 0.0007	1.2136 ± 0.0015	1.0722 ± 0.0014
Half mixture (ours)	1.2080 ± 0.0006	1.2159 ± 0.0008	1.0721 ± 0.0011
Random	1.2252 ± 0.0019	1.2450 ± 0.0015	1.0807 ± 0.0017

TABLE 9: English macro document BPC on 1,000 shared canonical documents, each truncated to at most 512 Unicode characters. Lower BPC is better. English results are descriptive only. Values are means ± sample standard deviation (SD) across three paired training seeds. All five predeclared initializers use the top-four training policy. The analogical initializer is a character-ngram proxy. No initializer is selected from these results. The held-out data had previously been inspected.

Model	Steps	Comparator	Δ BPC ↓	95% interval	Holm p
0.6B	2,000	Compositional mean	-0.0485	[-0.0509, -0.0463]	0.00119988
0.6B	2,000	Moment-matched mean	-0.1251	[-0.1305, -0.1192]	0.00119988
0.6B	2,000	Character-ngram analogical proxy	-0.0063	[-0.0080, -0.0043]	0.00119988
0.6B	2,000	Random	-0.9159	[-0.9278, -0.9041]	0.00119988
0.6B	5,087	Compositional mean	-0.0402	[-0.0420, -0.0384]	0.00119988
0.6B	5,087	Moment-matched mean	-0.0984	[-0.1036, -0.0923]	0.00119988
0.6B	5,087	Character-ngram analogical proxy	-0.0009	[-0.0028, +0.0010]	0.382562
0.6B	5,087	Random	-0.7355	[-0.7484, -0.7221]	0.00119988
1.7B	2,000	Compositional mean	-0.0439	[-0.0460, -0.0418]	0.00119988
1.7B	2,000	Moment-matched mean	-0.1474	[-0.1578, -0.1365]	0.00119988
1.7B	2,000	Character-ngram analogical proxy	-0.0117	[-0.0135, -0.0097]	0.00119988
1.7B	2,000	Random	-0.8741	[-0.8873, -0.8609]	0.00119988

TABLE 10: Arabic macro document BPC on 1,500 shared canonical documents, each truncated to at most 512 Unicode characters. Lower BPC is better. Half-mixture LEP (ours) minus each initializer comparator, with the top-four training policy. Negative differences favor the half mixture. Intervals are 95% marginal crossed seed/document bootstrap intervals over three paired training seeds and 1,500 shared documents; they are not simultaneous intervals. Holm correction covers all 12 planned initializer contrasts. The held-out data had previously been inspected.

α	Initializer	Arabic BPC ↓	English BPC ↓
0.00	Character-ngram analogical proxy	1.4174 ± 0.0014	1.2069 ± 0.0007
0.25	Quarter compositional	1.4052 ± 0.0007	1.2090 ± 0.0007
0.50	Half mixture (ours)	1.4111 ± 0.0003	1.2080 ± 0.0006
0.75	Three-quarters compositional	1.4289 ± 0.0006	1.2069 ± 0.0004
1.00	Compositional mean	1.4596 ± 0.0018	1.2087 ± 0.0008

TABLE 11: Arabic macro document BPC on 1,500 shared canonical documents, each truncated to at most 512 Unicode characters. Lower BPC is better. Values are means ± sample standard deviation (SD) across three paired training seeds. Complete exploratory five-point mixture sweep at 0.6B, 2,000 steps, top-four training. α weights the compositional component; $1-\alpha$ weights the character-ngram analogical proxy. Endpoints reuse the component runs. The predeclared primary value remains $\alpha = 0.5$. English uses 1,000 canonical documents with the same prefix rule and is descriptive. The held-out data had previously been inspected.

G. ARABIC TASK ACCURACY AND ENGLISH RETENTION

Table 14 includes all 49 task blocks for the fixed base and three seeds each of nominal-500M top-four LEP and CPT. Our LEP achieves the highest Arabic macro point estimate: 40.65%, compared with 39.58% for CPT and 39.20% for the base. Its mean accuracy exceeds CPT on MadinahQA (41.28% versus 38.31%), ArabicEXAMS (30.91% versus 29.73%) and AlGhafa (44.87% versus 39.44%). LEP also exceeds the base on these three tasks and AraTrust. The LEP–CPT macro difference is 1.07 percentage points, with a marginal 95% interval of $[-0.81, 2.94]$ and $p_{\text{Holm}} = 0.535$ across the nine downstream contrasts. The LEP–base macro difference is also unresolved. The task gains are not uniform: LEP's mean accuracy is lower than CPT on ArabicMMLU and AraTrust, and lower than the base on ArabicMMLU. Thus the panel shows favorable Arabic aggregate point estimates and benchmark-dependent changes, with uncertainty that does not establish an

Model	Steps	Trainability	CPT hours ↓	LEP hours ↓	CPT peak allocated GiB ↓	LEP peak allocated GiB ↓
0.6B	2,000	Top four	1.793	2.236	16.792	29.773
0.6B	2,000	Full	1.932	2.370	18.916	31.898
0.6B	5,087	Top four	4.530	5.654	16.792	29.773
0.6B	5,087	Full	4.881	6.013	18.916	31.898
1.7B	2,000	Top four	2.557	3.281	21.013	34.993
1.7B	2,000	Full	2.855	3.566	27.795	41.775

TABLE 12: Successful training resources on H100 80GB GPUs for the six predeclared CPT/half-mixture LEP (ours) comparisons. Hours are mean train_runtime per run over three training seeds, including in-loop validation/checkpoint work; initialization, model/data preparation, final-model save and separate evaluations are excluded. Device allocated memory is the maximum across the three runs, including resident model memory. These values are not total campaign cost: initialization and failed attempts are accounted separately, with missing timing endpoints for quarantined attempts. Matched nominal packed-token budgets do not imply equal actual tokens, text exposure or compute. Full pairs update every parameter; top-four policies differ in embedding-row updates.

Model	Batch	Mean end-to-end s/batch ↓	Generation tokens/s ↑	Generated characters/s ↑	Peak allocated GiB ↓	Peak reserved GiB ↓
0.6B Base	1	2.082	61.51	145.56	1.170	1.195
0.6B Base	4	2.243	228.46	543.60	1.259	1.344
0.6B CPT	1	2.082	61.53	147.67	1.170	1.195
0.6B CPT	4	2.244	228.35	555.14	1.259	1.348
0.6B LEP	1	2.079	61.61	243.61	1.415	1.439
0.6B LEP	4	2.256	227.09	864.41	1.489	1.582
1.7B Base	1	2.127	60.21	147.55	3.265	3.279
1.7B Base	4	2.271	225.67	564.12	3.353	3.445
1.7B CPT	1	2.249	56.95	138.58	3.265	3.279
1.7B CPT	4	2.454	208.78	504.77	3.353	3.445
1.7B LEP	1	2.300	55.67	212.71	3.759	3.771
1.7B LEP	4	2.475	206.99	763.33	3.832	3.912

TABLE 13: Descriptive serving measurements including LEP (ours) on one H100 80GB GPU, in the displayed model order. All models use the same 32 canonical Arabic prompts of 256 Unicode characters, 128 greedy new tokens, batches 1 and 4, two warmup batches per batch size, brain floating-point 16 (BF16), left padding and key–value (KV) caching, without end-of-sequence (EOS) stopping. Bases are fixed checkpoints rerun in this sequence; adapted checkpoints use predeclared training seed 1 (0.6B: 5,087 steps; 1.7B: 2,000 steps; top-four policy). Mean end-to-end seconds per batch equals total end-to-end time divided by 32 or 8 measured batches, respectively; it is not per-request latency. End-to-end includes tokenization, host/device transfers, generation and text decoding, excluding model loading. Throughput uses 4,096 generated tokens or the observed decoded Unicode-character count, respectively, divided by total generation time. Memory is maximum device allocated/reserved memory across batches, including the model. Generated content differs and can repeat after EOS; character throughput is content-dependent and is not quality-adjusted utility or equal-output latency. Full phase timings and generated content are retained in the raw records. No serving significance claim is made.

aggregate accuracy gain or equivalence. The controlled BPC improvement is the more consistent finding.

English macro accuracy is 62.92% for our LEP, 58.33% for CPT and 66.65% for the base. LEP exceeds CPT by 4.58 points (interval $[1.62, 7.45]$, $p_{\text{Holm}} = 0.0104$), but trails the base by 3.73 points (interval $[-5.77, -1.77]$, $p_{\text{Holm}} = 0.00250$). Both adapted methods have lower mean accuracy than the base on each English task. Canonical English BPC is 1.2159 for LEP, 1.2430 for CPT and 1.1941 for the base; all three pairwise BPC contrasts survive the declared correction. Thus LEP loses less English performance than CPT under these training policies, but it does not preserve the base model's performance. Table 15 gives the complete contrast family. Appendix E separates the corrected historical fixed-checkpoint diagnostics from this training-seed panel.

VI. DISCUSSION

We discuss tokenizer configuration, compression and adaptation cost, embedding initialization, and evaluation across vocabularies.[6]

[6]AI assistance: OpenAI Codex [3]; see Acknowledgment.

Endpoint	n	Base (fixed)	CPT	LEP
↑ ArabicMMLU	1,000	34.5000	34.9333 ± 1.1015	32.8000 ± 0.5292
↑ MadinahQA	977	38.1781	38.3146 ± 1.1274	41.2828 ± 0.5820
↑ ArabicEXAMS	537	29.2365	29.7331 ± 1.2399	30.9125 ± 0.9676
↑ AraTrust	387	53.2300	55.4694 ± 1.1935	53.4022 ± 1.2746
↑ AlGhafa	1,763	40.8395	39.4403 ± 1.0974	44.8667 ± 0.6003
↑ MMLU	1,000	51.8000	43.6000 ± 1.1533	47.4333 ± 0.1528
↑ ARC-Easy	1,000	81.5000	73.0667 ± 2.4420	78.4000 ± 1.1358
↑ Arabic macro	4,664	39.1968	39.5782 ± 1.0701	40.6529 ± 0.3328
↑ English macro	2,000	66.6500	58.3333 ± 1.7323	62.9167 ± 0.5774
↓ English BPC	1,000	1.1941	1.2430 ± 0.0014	1.2159 ± 0.0008

TABLE 14: Qwen3-0.6B with LEP (ours): fixed base and three training seeds per adapted arm at 5,087 steps, top-four policy. Base is one fixed checkpoint, not three independent runs. Adapted values are seed means ± sample standard deviation (SD). Task accuracies and task macros are percentages (higher is better); macros weight the five Arabic or two English tasks equally. n is the exact item count per task, summed item count for each macro, or document count for BPC. English BPC uses canonical prefixes of at most 512 Unicode characters (lower is better). All 49 checkpoint/task blocks are included. The held-out data had previously been inspected.

Endpoint	Contrast	Units	Difference	95% interval	Holm p
↑ Arabic macro	LEP-CPT	percentage points	+1.0747	[-0.8117, +2.9381]	0.534547
↑ Arabic macro	LEP-BASE	percentage points	+1.4560	[-0.3139, +3.2868]	0.336866
↑ Arabic macro	CPT-BASE	percentage points	+0.3813	[-1.3659, +2.1642]	0.681932
↑ English macro	LEP-CPT	percentage points	+4.5833	[+1.6163, +7.4500]	0.010399
↑ English macro	LEP-BASE	percentage points	-3.7333	[-5.7667, -1.7667]	0.00249975
↑ English macro	CPT-BASE	percentage points	-8.3167	[-10.7500, -5.9167]	0.00089991
↓ English BPC	LEP-CPT	BPC	-0.0270	[-0.0294, -0.0246]	0.00089991
↓ English BPC	LEP-BASE	BPC	+0.0219	[+0.0201, +0.0236]	0.00089991
↓ English BPC	CPT-BASE	BPC	+0.0489	[+0.0463, +0.0514]	0.00089991

TABLE 15: All nine predeclared downstream contrasts including LEP (ours); differences are left minus right. Positive accuracy differences (percentage points) and negative BPC differences favor the left arm. 95% marginal bootstrap intervals share three paired training-seed draws and resample paired items within the seven fixed tasks, without task resampling; English BPC uses crossed paired seed/document resampling on 1,000 documents. Base is held fixed in every draw. Holm correction covers all nine contrasts; intervals are not simultaneous. Nonsignificance does not establish equivalence. The held-out data had previously been inspected.

A. WHY UNIGRAM AND NORMALIZATION SUIT ARABIC

Unigram assigns probabilities to alternative segmentations and prunes pieces during vocabulary learning. Normalization changes the strings on which those pieces are learned. In Table 4, our tested Unigram configuration has the lowest fertility among the tested configurations. This comparison supports the reported configuration choice, but does not isolate morphological alignment or establish a complementary mechanism between normalization and segmentation. Unknown-token behavior also differs across these configurations and must be considered alongside fertility.

B. COMPRESSION AND ADAPTATION COST

Token compression and language-model quality are separate outcomes. Encoding the same text in fewer tokens can increase the text covered by a fixed training-token budget. It does not establish an equal-compute comparison: vocabulary expansion changes the embedding and output layers, and training cost depends on the trainable parameters and execution setup. Likewise, characters per token on an input corpus do not measure generated characters per second. The adaptation results must therefore be interpreted with their text coverage, trainable policy and measured resource use stated separately.

C. NORM AND DIRECTION OF A NEW EMBEDDING

Proposition 1 describes expected squared distance from the donor mean under its moment assumptions. This centered quantity is not an absolute row norm or a bound on the change in the model's predictions. Proposition 2 gives the squared error of the fixed average relative to a reference row. It is no larger than the mean component error, but improving on both components requires the stated additional condition. The reference row and component errors are unobserved here, so the identity does not establish error cancellation or an improvement in language-model loss. The initializer comparison evaluates that empirical question under specified training conditions.

D. EVALUATION ACROSS VOCABULARIES

Per-token losses from different tokenizers refer to different prediction units. A BPC comparison requires identical declared character spans and consistent conditioning and scoring rules. Task evaluation likewise requires checking the actual candidate strings under every tokenizer and retaining all answer options. Rotating options across items counterbalances their presentation; it does not establish that a model's position preference has been removed. Differences in task point estimates must be reported with their uncertainty and do not by themselves establish task preservation.

VII. CONCLUSION

We summarize the findings, then delimit their scope and describe artifact availability.[7]

Our AraToken combines Arabic normalization with Unigram segmentation to reduce the tokens needed for the tested Arabic corpora. LEP transfers that vocabulary to Qwen3, and the controlled study finds lower canonical-Arabic BPC than CPT across both model sizes and both trainability policies. The fixed initializer mixture improves on its components in two settings, with an unresolved advantage over analogical initialization at the larger 0.6B budget. The sensitivity study does not identify equal weighting as optimal.

In the task panel, LEP has the highest Arabic macro point estimate and higher mean accuracy than CPT on three of five Arabic benchmarks. It also retains more English accuracy and likelihood than CPT. These results complement the consistent Arabic likelihood advantage, with an unresolved Arabic aggregate accuracy difference and English performance below the unadapted base. LEP requires more training time and memory, so its token compression advantage must be distinguished from computational cost. Future work can examine dialect-specific training and replay of English data; neither intervention is evaluated here.

LIMITATIONS

The controlled study uses three seeds, two Qwen3 base sizes and previously inspected evaluation data. Bootstrap intervals approximate uncertainty with limited training-seed coverage; the task panel fixes five Arabic and two English tasks and does not measure task accuracy at 1.7B or with full training. Our initializer baseline is a character-ngram proxy rather than published WECHSEL or FOCUS, and the algebra does not guarantee empirical improvement. Our top-four LEP masks original embedding rows whereas CPT updates them; the full-training controls address parameter freedom but do not equalize vocabulary capacity, character exposure or FLOPs.

AraToken is trained on Modern Standard Arabic. A separate byte-fallback variant reduces tokens per word by 21.8% on dialectal tweets, a smaller gap than the normalized main comparison. Normalization can remove case and orthographic distinctions. The MinHash diagnostic does not certify absence of exact or near-duplicate training/test content. Serving uses one fixed training seed per adapted architecture, and fixed-token greedy outputs can be repetitive; generated-character throughput is not a quality-adjusted latency measure. Incomplete quarantined-attempt timing records limit total campaign-cost accounting.

DATA AND CODE AVAILABILITY

The previously released code, AraToken tokenizers and identifiers of the `dev` and `test` documents are available at https://github.com/mariklolik/Aratoken. The four document splits of FineWeb2-HQ used for tokenizer training, adaptation and evaluation are available at https://huggingface.co/datasets/mariklolik/AraToken-FineWeb2-HQ-ar. All tokenizers, including the extended Qwen3 tokenizers, are available at https://huggingface.co/mariklolik/AraToken, and the adapted Qwen3-0.6B and Qwen3-1.7B checkpoints with their continued-pretraining baselines at https://huggingface.co/mariklolik. The accompanying revision packet contains the controlled-run measurements, individual seed results, execution configuration and artifact hashes; these follow-up artifacts are distinct from the historical releases. The new final-step model weights are retained in the authors' compute environment and are not included in the public releases or revision packet.

ACKNOWLEDGMENT

OpenAI Codex [3] assisted with the revision of the Abstract, Introduction, Related Work, Methodology, Experimental Setup, Results, Discussion, Conclusion and appendices; code corrections, experiment orchestration, statistical checks and table generation; bibliography verification; and the overview, normalization-example and LEP-architecture diagrams. The assistance included substantive drafting and code generation rather than language editing alone. Numerical results were obtained by executing the recorded experiments and checks on the stated FSK hardware; the revision packet retains the AI-assistance ledger and raw evidence.

APPENDIX A EVALUATION PROTOCOL

This appendix specifies split membership, duplicate-screening limits and metric aggregation.[8]

SPLIT CONSTRUCTION

For each corpus ID, we compute an eight-byte BLAKE2b digest of its UTF-8 representation, interpret it as a big-endian integer and take the remainder modulo 1000. Half-open bucket ranges [0,600), [600,900), [900,950)

[7] AI assistance: OpenAI Codex [3]; see Acknowledgment.

[8] AI assistance: OpenAI Codex [3]; see Acknowledgment.

and [950,1000) define language-model training, tokenizer training, development and test. The manifest records every third source shard (29 selected paths out of 85), minimum length 200 characters and no additional positive quality-score cutoff. Character budgets stop collection after an accepted document, so stored totals can slightly exceed their targets. Split assignment is deterministic, but multithreaded collection and budget stopping do not by themselves guarantee identical selected rows or order on a new download; the released files, ID lists and hashes define the actual study data.

Stored document/character totals are: language-model training 662,685/2,000,000,759; tokenizer training 159,826/480,001,125; development 2,522/8,000,072; test 6,602/20,001,350. The manifest's approximate token counts are character-based estimates, not measured tokenizer output counts.

NEAR DUPLICATES

The historical MinHash overlap diagnostic [50] indexes both training splits using word five-grams and 112 MinHash permutations arranged as 14 bands of eight. It reports locality-sensitive hashing (LSH) candidates for 43 of 6,602 test documents against 822,511 training documents. Candidate pairs were not verified by exact Jaccard similarity, so this is a candidate-hit rate, not a measured count above a hard 0.72 similarity cutoff. The recorded zero exact-duplicate count concerns repeated text within the test set only; the diagnostic does not establish absence of exact train–test duplicates. No deduplication intervention or claim of zero content leakage follows from this check.

MEASUREMENT CONVENTIONS

The historical CAMeL record identifies the first 500 documents and the corpus-ratio formula, but does not preserve the exact integer morpheme denominator or analyzer revision. This limits exact reproduction of that diagnostic.

Fertility and compression are per-document means with 95% bootstrap intervals over 10,000 resamples. We also compute corpus-level aggregates, but a table never mixes the two conventions. In the algorithm comparison, each tokenizer is measured under the preprocessing recorded in its saved specification: BPE and WordPiece apply their normalizer inside the tokenizer, and SentencePiece receives normalized text.

APPENDIX B REGISTER AND BYTE FALLBACK

Table 16 reports fertility on three kinds of text: the `test` split, 1,650 documents in the stored corpus's lowest quality quartile, and 2,089 dialectal tweets from ArSarcasm [51]. The low-quality subset is defined by the source quality-score quartile within the stored test corpus. This score is not a register annotation, and the comparison does not establish that all low-scoring documents are Modern Standard Arabic or explain their tokenizer behavior causally. On dialectal tweets every tokenizer needs more tokens per word, and our AraToken keeps the lowest fertility of all systems.[9]

Tokenizer	MSA ↓	low-q ↓	dialectal ↓	ratio
Qwen3 / Qwen2.5	2.180	2.180	3.205	1.47
AraBERT	1.835	1.838	3.003	1.64
AraToken, 95% coverage	1.281	1.282	2.563	2.00
AraToken, 150K, Alif unified	1.208	1.207	2.332	1.93
AraToken, 150K	1.213	1.212	2.359	1.94
AraToken, 150K, byte fallback	1.216	1.215	2.506	2.06

TABLE 16: Fertility of AraToken (ours) and baselines by register, each under its own normalization. MSA denotes Modern Standard Arabic: a 3,000-document `test` sample. Low-q is the lowest quality quartile (1,650 of the 6,602 stored documents), and dialectal 2,089 ArSarcasm tweets; ratio is dialectal over MSA.

A tokenizer without complete character coverage can replace unsupported spans with an unknown token, lowering its token count while losing content. The mean per-document unknown-token fractions on the tweets are 5.1% for AraToken and 3.8% for AraBERT. With byte fallback, AraToken encodes such a character as its UTF-8 bytes. Its dialectal fertility is then 2.51 against 3.21 for the tokenizer of Qwen3, 21.8% lower, and every character is encoded. On the 3,000-document test sample, byte fallback raises fertility by 0.3%, from 1.213 to 1.216. The separate word-level unknown rate on the first 500 test documents falls from 0.31% to zero. Inside LEP the byte-level vocabulary of Qwen3 covers every character, so the extended model has no unknown tokens in either case.

APPENDIX C WHAT PRUNING REMOVES

Of the 28,443 Arabic pieces that the full vocabulary holds and the 99%-coverage vocabulary drops, 89% have five or more characters, with a mean length of 6.4. Table 17 quantifies the token-count cost of our pruned vocabularies by frequency band. At 99% coverage fertility does not change on the thousand most frequent words and rises by about 0.7%, 1.8% and 4.2% in the lower frequency bands. At 95% coverage the rise is 0.4% on the most frequent words and 15.8% on rare and unseen words. These thresholds provide explicit vocabulary-size versus token-count choices; task quality under pruning requires separate evaluation.[10]

APPENDIX D TRAINABLE SET AND ADAPTATION BASELINES

Table 18 retains the historical trainable-layer diagnostics for our mean-initialized LEP. An additional configuration trained only the new rows, with every transformer layer frozen, and reached 1.5575 BPC. These configurations

[9]AI assistance: OpenAI Codex [3]; see Acknowledgment.
[10]AI assistance: OpenAI Codex [3]; see Acknowledgment.

Frequency band	words	full ↓	99% ↓	95% ↓	Δ ↓
top 1k	127,890	1.015	1.015	1.018	+0.4%
1k–10k	77,258	1.118	1.126	1.186	+6.0%
10k–100k	60,128	1.357	1.381	1.506	+11.0%
rare / unseen	34,724	2.317	2.415	2.683	+15.8%

TABLE 17: Fertility of AraToken (ours) over the first 300,000 word occurrences in the `test` split, with frequency bands ranked on the first 40,000 `lm-train` documents. Frequency ranks and test words use Alif-unifying normalization with Alif-preserving tokenizer files. Δ compares the full vocabulary with 95% coverage.

Unfrozen layers	BPC ↓
L27 only	1.4176 ±0.0003
L24–27	1.3810 ±0.0009
L18–27	1.3232 ±0.0014
L9–27	1.2918 ±0.0007
all layers	1.2474 ±0.0008

TABLE 18: Historical within-tokenizer BPC diagnostics of mean-initialized LEP (ours) by trainable set, mean ± standard deviation (SD) over three seeds, 2,000 steps. All rows use the legacy 1,024-token-prefix scoring rule.

share the same extended tokenizer and historical scoring rule: a separately tokenized prefix of up to 1,024 tokens per document. Their within-tokenizer comparison is distinct from the corrected common-character BPC comparisons in Section V-D; the absolute values should not be compared across these protocols.[11]

APPENDIX E TASK EVALUATION

We first define choice scoring, then report corrected historical OALL and classification diagnostics and English likelihood across training policies.[12]

SCORING PROTOCOL

The data-only preflight fixes the ordered task items, options, gold labels, exclusions and source fingerprints before new adapted-model scoring. We retain the first 1,000 raw rows per configuration before validity filtering, without backfilling. The resulting Arabic task counts are 1,000, 977, 537, 387 and 1,763 in the order listed above (4,664 total); each English task contains 1,000 items. Dataset configurations and splits are recorded with the accompanying cohort receipt. AraTrust uses its named training split as an evaluation source; none of these task data is used for adaptation.

Zero-shot inference uses BF16 and deterministic argmax over letter continuations. The continuation score is summed log probability divided by raw UTF-8 byte length, including its leading space; prompt and continuation IDs are encoded separately and concatenated. Option rotation is the accepted item index modulo the number of options, restarting for each task. This is a fixed scoring convention, not proof that position bias is canceled; it introduces no independent evaluation-seed replication. The candidate audit verifies the actual continuation encodings. Arabic letters are single tokens under both tokenizers; English A–E are also single tokens, but the extended tokenizer lowercases them internally. Raw inputs are shared while tokenizer-internal normalization is preserved.

The base is one fixed reference and is reused in each adapted-seed contrast. The accompanying evaluation artifacts identify the ordered cohort, gold labels, scoring protocol and model/tokenizer hashes. Each adapted checkpoint is checked against its final-step training record; the same model is used across all seven tasks. Historical classification split/demonstration seeds are evaluation seeds, not independent model-training seeds.

HISTORICAL OALL AT THE ITEM LEVEL

Corrected evaluation of the historical fixed checkpoints uses 4,664 valid items. Pooled accuracy is 0.3915 for our mean-initialized LEP, 0.3861 for the unadapted model and 0.3801 for CPT. The LEP–CPT difference is 1.14 percentage points, with a paired-item 95% interval of $[-0.39, 2.62]$ points and Holm-adjusted $p = 0.449$ across the three pooled comparisons. No pooled comparison rejects the null at 0.05. LEP has lower point accuracy than CPT on AlGhafa and lower point accuracy than the base on ArabicMMLU and AlGhafa. These fixed-checkpoint diagnostics do not establish preservation, and their historical CPT trainability is not the new controlled comparison.

HISTORICAL CLASSIFICATION AND INFERENCE

Table 19 reports Macro F1 on ASTD [52], ArSarcasm [51], SANAD [53] and XNLI-ar [54] for the historical fixed checkpoints. The 18 SANAD evaluations were rerun with all seven answer labels available; the other 72 prediction files are unchanged. Means summarize three evaluation seeds, not independently trained models. The label-prior reference samples predictions from the gold label distribution and is descriptive. Two model–task–shot cells exceed it: CPT on binary sarcasm without demonstrations (0.509 versus 0.497), and our historical LEP on seven-class topic classification with three demonstrations (0.152 versus 0.144). The remaining task–shot settings do not exceed that reference for any model. These results show limited task utility under this historical protocol; exceeding the reference does not establish statistical significance or consistent superiority. The new training-seed task panel is reported separately.

ENGLISH LIKELIHOOD ACROSS TRAINING POLICIES

Table 6 reports English BPC for every controlled pair of our LEP and CPT. Fully trainable models have higher

[11] AI assistance: OpenAI Codex [3]; see Acknowledgment.
[12] AI assistance: OpenAI Codex [3]; see Acknowledgment.

Benchmark (Macro F1) ↑	Chance	Base	CPT	LEP
0-shot				
ASTD (4-class)	0.245	0.206	0.207	0.197
ArSarcasm (3-class)	0.329	0.304	0.299	0.319
ArSarcasm-Sarcasm (2-class)	0.497	0.424	**0.509**	0.423
SANAD (7-class)	0.143	0.130	0.127	0.133
XNLI-ar (3-class)	0.335	0.330	0.315	0.318
3-shot				
ASTD	0.250	0.187	0.207	0.158
ArSarcasm	0.336	0.198	0.215	0.296
ArSarcasm-Sarcasm	0.499	0.481	0.336	0.441
SANAD	0.144	0.139	0.112	**0.152**
XNLI-ar	0.336	0.209	0.289	0.245

TABLE 19: Historical fixed-checkpoint Macro F1 on Arabic classification and inference tasks, mean over three evaluation seeds, including LEP (ours). The Chance column is the Macro F1 of a predictor that draws labels from the gold label distribution, and bold cells exceed this descriptive reference, without a significance claim. Options are scored by their letter, with the option order rotated from item to item.

English BPC than their corresponding top-four models in all three settings for both LEP and CPT. These additional BPC values are descriptive; the predeclared English inference and task-accuracy comparisons concern the nominal-500M top-four panel in Table 15. The old tokenizer-specific truncated English values are retained in the original artifacts and are not used as common-text comparisons.

APPENDIX F TRAINING DETAILS

This appendix records the training configuration, parameter counts and serving protocol.[13]

The new runs use one GPU per training process, a per-device microbatch of eight sequences and twelve accumulation steps, giving a nominal global batch of 96. Sequence length is 1,024, BF16 and gradient checkpointing are enabled, incomplete dataloader batches are dropped, and evaluation/saving occur every 1,000 steps in addition to the initial-step diagnostic evaluation. The run environment records PyTorch 2.5.1+cu124, Transformers 4.51.3, Tokenizers 0.21.4 and Datasets 4.0.0. Actual GPU identity and resource measurements accompany each run; the frozen FSK40 hardware receipt identifies H100 80GB HBM3.

The released packed streams contain 437,835 training and 1,734 validation blocks for the extended tokenizer, versus 783,751 and 3,120 for the base tokenizer. Thus their stored training sizes are 448,343,040 and 802,561,024 tokens. Preparation tokenizes without Python-side normalization, appends EOS, concatenates and packs full blocks, and drops the remainder of each packing batch. Tokenizer-internal normalization remains active. The larger token budget exceeds the extended stream's stored length, so the training schedule revisits it. EOS/padding masking by the training collator and partial accumulation groups mean nominal input counts must not be described as unique text exposure or prediction-target counts.

[13]AI assistance: OpenAI Codex [3]; see Acknowledgment.

Parameter	Value
Base models	Qwen3-0.6B-Base / Qwen3-1.7B-Base
Optimizer	AdamW, weight decay 0.01
Learning rate	2×10^{-4}, linear schedule
Warmup ratio	0.1
Global batch	96 sequences × 1,024 tokens
Steps	2,000 / 5,087 (nominal 500M tokens)
Trainable layers	24–27 unless stated otherwise
Original rows	masked in primary LEP; updated in CPT/full controls
Training seeds	1, 2, 3
Checkpoint	final completed step
Gradient checkpointing	enabled

TABLE 20: Hyperparameters of the new LEP (ours) and CPT adaptation runs. The primary policies update blocks 24–27; fully trainable controls update all parameters.

Model/policy	Total (M)	Updated (M)
0.6B CPT, top four	596.050	218.506
0.6B LEP, top four	730.081	197.229
0.6B CPT, full	596.050	596.050
0.6B LEP, full	730.081	730.081
1.7B CPT, top four	1720.575	512.509
1.7B LEP, top four	1988.638	469.954
1.7B CPT, full	1720.575	1720.575
1.7B LEP, full	1988.638	1988.638

TABLE 21: New-run parameter counts for LEP (ours) and CPT, in millions. Updated counts exclude masked original rows; tied input/output embeddings count once. Both full-training controls update every parameter. Peak memory is measured separately.

Table 20 summarizes the settings; Table 21 distinguishes total and effectively updated parameters.

SERVING MEASUREMENTS

The descriptive serving panel fixes six checkpoints before their new outcomes: the 0.6B base, nominal-500M CPT and hybrid seed 1, followed by the 1.7B base, CPT and hybrid seed 1. All use the same exclusively leased H100 80GB GPU and the same 32 canonical Arabic prompts with 256-character prefixes. Decoding uses BF16, left padding, KV cache, two warmup batches, batch sizes 1 and 4, and 128 greedy new tokens without EOS stopping. Bases are measured anew in this sequence. Prompt and artifact hashes, generated text/tokens, raw batch timings and device/software metadata are retained. Fixed-token character throughput depends on generated content and does not measure equal-output latency or

quality-adjusted utility. The panel has no task-quality or inferential significance claim.

The historical training traces and the separate plot of Table 4 are retained with the accompanying artifacts.

REFERENCES

[1] P. Rust, J. Pfeiffer, I. Vulic, S. Ruder, and I. Gurevych, "How good is your tokenizer? on the monolingual performance of multilingual language models," in Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics, pp. 3118–3135, 2021.

[2] A. Petrov, E. La Malfa, P. Torr, and A. Bibi, "Language model tokenizers introduce unfairness between languages," Advances in Neural Information Processing Systems, vol. 36, 2023.

[3] OpenAI, "Codex." https://openai.com/codex/. Accessed: Nov. 2, 2026.

[4] N. Y. Habash, Introduction to Arabic natural language processing. Springer, 2010.

[5] A. Alhumud, A. Alhammadi, and M. B. Khan, "Arabicnumbench: Evaluating arabic number reading in large language models," 2026.

[6] S. Yagi, A. Elnagar, and E. Yaghi, "Arabic punctuation dataset," Data in Brief, vol. 53, p. 110118, feb 2024.

[7] W. Antoun, F. Baly, and H. Hajj, "Arabert: Transformer-based model for arabic language understanding," arXiv preprint arXiv:2003.00104, 2020.

[8] N. Sengupta et al., "Jais and jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models," arXiv preprint arXiv:2308.16149, 2023.

[9] K. Hennara, S. Chrouf, M. M. Hamed, Z. Aldallal, O. Hadid, and S. AlModhayan, "Kuwain 1.5b: An arabic slm via language injection," 2025.

[10] J. Zhu, H. Huang, Z. Lin, J. Liang, Z. Tang, K. Almubarak, A. Alharthik, B. An, J. He, X. Wu, F. Yu, J. Chen, Z. Ma, Y. Du, H. Zhang, E. A. Alghamdi, L. Zhang, R. Sun, H. Li, B. Wang, and J. Xu, "Second language (Arabic) acquisition of LLMs via progressive vocabulary expansion," 2024.

[11] J. Jaber and O. Jaber, "RightNow-Arabic-0.5B-Turbo: An open sub-1B arabic language model via vocabulary injection and edge-first deployment," 2026.

[12] A. Tejaswi, N. Gupta, and E. Choi, "Exploring design choices for building language-specific LLMs," in Findings of the Association for Computational Linguistics: EMNLP 2024, pp. 10485–10500, 2024.

[13] B. Minixhofer, F. Paischer, and N. Rekabsaz, "Wechsel: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models," in Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics, pp. 3992–4006, 2022.

[14] K. Dobler and G. de Melo, "Focus: Effective embedding initialization for monolingual specialization of multilingual models," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 13440–13454, 2023.

[15] Q. Team, "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025.

[16] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, pp. 1715–1725, 2016.

[17] M. Schuster and K. Nakajima, "Japanese and korean voice search," in 2012 IEEE international conference on acoustics, speech and signal processing (ICASSP), pp. 5149–5152, IEEE, 2012.

[18] T. Kudo, "Subword regularization: Improving neural network translation models with multiple subword candidates," arXiv preprint arXiv:1804.10959, 2018.

[19] T. Kudo and J. Richardson, "Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 66–71, 2018.

[20] S. R. Vemula, S. Dandapat, D. M. Sharma, and P. Krishnamurthy, "Rethinking tokenization for rich morphology: The dominance of unigram over bpe and morphological alignment," in Proceedings of the IJCNLP-AACL 2025 Student Research Workshop, 2025.

[21] K. Darwish, N. Habash, M. Abbas, H. Al-Khalifa, H. T. Al-Natsheh, H. Bouamor, K. Bouzoubaa, V. Cavalli-Sforza, S. R. El-Beltagy, W. El-Hajj, M. Jarrar, and H. Mubarak, "A panoramic survey of natural language processing in the arab world," Communications of the ACM, vol. 64, no. 4, pp. 72–81, 2021.

[22] O. Obeid, N. Zalmout, S. Khalifa, D. Taji, M. Oudah, B. Alhafni, G. Inoue, F. Eryani, A. Erdmann, and N. Habash, "Camel tools: An open source python toolkit for arabic natural language processing," in Proceedings of the Twelfth Language Resources and Evaluation Conference, pp. 7022–7032, 2020.

[23] M. Kamal Eddine, N. Tomeh, N. Habash, J. Le Roux, and M. Vazirgiannis, "Arabart: a pretrained arabic sequence-to-sequence model for abstractive summarization," in Proceedings of the Seventh Arabic Natural Language Processing Workshop (WANLP), pp. 31–42, 2022.

[24] E. M. B. Nagoudi, A. Elmadany, and M. Abdul-Mageed, "Arat5: Text-to-text transformers for arabic language generation," arXiv preprint arXiv:2109.12068, 2022.

[25] H. Huang et al., "Acegpt, localizing large language models in arabic," arXiv preprint arXiv:2309.12053, 2023.

[26] M. S. Bari, Y. Alnumay, N. A. Alzahrani, N. M. Alotaibi, H. A. Alyahya, S. AlRashed, F. A. Mirza, S. Z. Alsubaie, H. A. Alahmed, G. Alabduljabbar, R. Alkhathran, Y. Almushayqih, R. Alnajim, S. Alsubaihi, M. Al Mansour, M. Alrubaian, A. Alammari, Z. Alawami, A. Al-Thubaity, A. Abdelali, J. Kuriakose, A. Abujabal, N. Al-Twairesh, A. Alowisheq, and H. Khan, "ALLaM: Large language models for arabic and english," in The Thirteenth International Conference on Learning Representations, 2025.

[27] A. AlOtaibi, L. Alyahya, R. Alshabanah, S. Alfawzan, S. Alarefei, R. Alsabti, N. Alsubaie, A. Alhuzaymi, L. Alkhelb, M. Alsayari, W. Alahmed, O. Talabay, J. Alowibdi, S. Alelyani, and A. Bibi, "Tahakom llm guidelines and recipes: From pre-training data to an arabic llm," 2025.

[28] M. Artetxe, S. Ruder, and D. Yogatama, "On the cross-lingual transferability of monolingual representations," in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 4623–4637, 2020.

[29] Z.-X. Yong, H. Schoelkopf, N. Muennighoff, A. F. Aji, D. I. Adelani, K. Almubarak, M. S. Bari, L. Sutawika, J. Kasai, A. Baruwa, et al., "Bloom+1: Adding language support to bloom for zero-shot prompting," in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pp. 11682–11703, 2023.

[30] J. Pfeiffer, I. Vulić, I. Gurevych, and S. Ruder, "Mad-x: An adapter-based framework for multi-task cross-lingual transfer," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, pp. 7654–7673, 2020.

[31] J. Zhao, Z. Zhang, L. Gao, Q. Zhang, T. Gui, and X. Huang, "Llama beyond english: An empirical study on language capability transfer," arXiv preprint arXiv:2401.01055, 2024.

[32] H. Han, A. I. Eriguchi, H. Xu, H. Hoang, M. Carpuat, and H. Khayrallah, "Adapters for altering LLM vocabularies: What languages benefit the most?," in Proceedings of the Thirteenth International Conference on Learning Representations (ICLR), 2025.

[33] J. Hewitt, "Initializing new word embeddings for pretrained language models." https://www.cs.columbia.edu/~johnhew/vocab-expansion.html, 2021. Accessed 2026-11-02.

[34] C. M. Downey, T. Blevins, N. Goldfine, and S. Steinert-Threlkeld, "Embedding structure matters: Comparing methods to adapt multilingual vocabularies to new languages," in Proceedings of the 3rd Workshop on Multilingual Representation Learning (MRL), 2023. arXiv:2309.04679.

[35] R. Joshi, U. Vaidya, S. S. Chauhan, and N. Wartikar, "Beyond initialization loss: A systematic study of token embedding initialization strategies for llm vocabulary extension," arXiv preprint arXiv:2608.03494, 2026.

[36] A. Yamaguchi, A. Villavicencio, and N. Aletras, "How can we effectively expand the vocabulary of LLMs with 0.01GB

of target language text?," Computational Linguistics, 2026. arXiv:2406.11477.
[37] N. Mundra, A. Khandavally, R. Dabre, R. Puduppully, A. Kunchukuttan, and M. M. Khapra, "An empirical comparison of vocabulary expansion and initialization approaches for language models," in Proceedings of the 28th Conference on Computational Natural Language Learning (CoNLL), pp. 84–104, 2024.
[38] A. Moi and N. Patry, "Huggingface's tokenizers: Fast state-of-the-art tokenizers optimized for research and production." https://github.com/huggingface/tokenizers, 2023. Version 0.13.3.
[39] B. Messmer, V. Sabolčec, and M. Jaggi, "Enhancing multilingual LLM pretraining with model-based data selection." arXiv:2502.10361, 2025. Dataset: https://huggingface.co/datasets/epfml/FineWeb2-HQ.
[40] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in International Conference on Learning Representations (ICLR), 2017.
[41] A. El Filali, M. ALOUI, T. Husaain, A. Alzubaidi, B. E. A. Boussaha, R. Cojocaru, C. Fourrier, N. Habib, and H. Hacid, "Open arabic llm leaderboard 2." https://huggingface.co/spaces/OALL/Open-Arabic-LLM-Leaderboard, 2025.
[42] F. Koto, H. Li, S. Shatnawi, J. Doughman, A. B. Sadallah, A. Alraeesi, K. Almubarak, Z. Alyafeai, N. Sengupta, S. Shehata, N. Habash, P. Nakov, and T. Baldwin, "ArabicMMLU: Assessing massive multitask language understanding in Arabic," in Findings of the Association for Computational Linguistics: ACL 2024, 2024.
[43] M. Hardalov, T. Mihaylov, D. Zlatkova, Y. Dinkov, I. Koychev, and P. Nakov, "EXAMS: A multi-subject high school examinations dataset for cross-lingual and multilingual question answering," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, pp. 5427–5444, 2020.
[44] E. A. Alghamdi, R. I. Masoud, D. Alnuhait, A. Y. Alomairi, A. Ashraf, and M. Zaytoon, "AraTrust: An evaluation of trustworthiness for LLMs in Arabic," arXiv preprint arXiv:2403.09017, 2024.
[45] E. Almazrouei, R. Cojocaru, M. Baldo, Q. Malartic, H. Alobeidli, D. Mazzotta, G. Penedo, G. Campesan, M. Farooq, M. Alhammadi, J. Launay, and B. Noune, "AlGhafa evaluation benchmark for Arabic language models," in Proceedings of ArabicNLP 2023, pp. 244–275, 2023.
[46] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in International Conference on Learning Representations, 2021.
[47] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? try ARC, the AI2 reasoning challenge," arXiv preprint arXiv:1803.05457, 2018.
[48] B. Efron, "Bootstrap methods: Another look at the jackknife," The Annals of Statistics, vol. 7, no. 1, pp. 1–26, 1979.
[49] S. Holm, "A simple sequentially rejective multiple test procedure," Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979.
[50] A. Z. Broder, "On the resemblance and containment of documents," in Proceedings of the Compression and Complexity of Sequences, pp. 21–29, 1997.
[51] I. Abu Farha and W. Magdy, "From Arabic sentiment analysis to sarcasm detection: The ArSarcasm dataset," in Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT), pp. 32–39, 2020.
[52] M. Nabil, M. Aly, and A. Atiya, "ASTD: Arabic sentiment tweets dataset," in Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, pp. 2515–2519, 2015.
[53] O. Einea, A. Elnagar, and R. Al Debsi, "SANAD: Single-label Arabic news articles dataset for automatic text categorization," Data in Brief, vol. 25, p. 104076, 2019.
[54] A. Conneau, R. Rinott, G. Lample, A. Williams, S. R. Bowman, H. Schwenk, and V. Stoyanov, "XNLI: Evaluating cross-lingual sentence representations," in Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 2475–2485, 2018.

MARK KASHIRSKIY received the B.S. degree in computer science from Novosibirsk State University in 2025. He is pursuing M.S. degrees at HSE University and ITMO University. His research interests include large language models, multilingual natural language processing, and efficient training.

ARTIOM LIPINSKI received the B.S. degree in computer science from Saint Petersburg State University. His research interests include Arabic natural language processing, domain adaptation, and cross-lingual transfer.

ILYA MAKAROV holds a Specialist degree in mathematics from Lomonosov Moscow State University and a Ph.D. in computer science from the University of Ljubljana. His research covers machine learning, computer vision, and natural language processing.